



Global Solution - 1º Semestre - 2026 – 2TIAPY

Tecnólogo em Inteligência Artificial

Disciplina: Generative AI & Advanced Nets

Grupo: Paulo Gabriel Pessoa da Silva – RM: 566446

William Stahl Sanches Furquim Garcia – RM: 562800

SpaceRAG

Assistente Inteligente com RAG para Dados e Documentos Espaciais

1. PROBLEMA ESCOLHIDO

1.1 Contexto

A nova economia espacial gera volumes crescentes de documentos técnicos, relatórios científicos, boletins climáticos e dados de missões. Agências como NASA, ESA e INPE publicam centenas de documentos mensalmente sobre satélites, monitoramento ambiental, agricultura inteligente e desastres naturais.

O problema central é o acesso eficiente à informação: especialistas e estudantes precisam consultar documentos extensos manualmente para encontrar respostas específicas, o que é ineficiente e sujeito a falhas humanas.

1.2 Solução Proposta

Desenvolvemos o SpaceRAG, um assistente inteligente que permite ao usuário:

1. Carregar documentos (PDF, TXT, DOCX) sobre temas espaciais
2. Fazer perguntas em linguagem natural
3. Receber respostas geradas com base no conteúdo dos documentos, com citação de fontes

2. ARQUITETURA DA SOLUÇÃO

2.1 Diagrama de Arquitetura

A solução é dividida em três camadas principais:

- Frontend: Interface web com tema dark space, suporte a drag-and-drop para upload
- Backend: FastAPI com processamento de documentos, embeddings e RAG
- Infraestrutura: FAISS para armazenamento vetorial e Ollama para geração local

2.2 Fluxo de Ingestão de Documentos

O processamento de documentos segue as etapas:

4. Document Loader: Suporte a PDF, DOCX e TXT com detecção automática de encoding
5. Chunking Inteligente: Divisão em chunks de 600 caracteres com sobreposição de 80 caracteres
6. Embedding Generation: Uso do modelo all-MiniLM-L6-v2 com 384 dimensões
7. FAISS Storage: Persistência de vetores em disco com busca por cosine similarity

2.3 Fluxo de Query RAG

Quando o usuário faz uma pergunta, o sistema:

8. Gera embedding da query com o mesmo modelo de embeddings
9. Busca os top-5 chunks mais relevantes no FAISS
10. Formata um prompt com contexto e query
11. Gera resposta usando Ollama (llama3.2) com temperature 0.2

3. FERRAMENTAS UTILIZADAS

Componente	Tecnologia	Versão	Finalidade
Backend	FastAPI + Uvicorn	0.111.0	API REST com endpoints para RAG
Embeddings	sentence-transformers	3.0.1	Modelo all-MiniLM-L6-v2
Vector Store	FAISS (faiss-cpu)	latest	Busca vetorial persistente
LLM	Ollama (llama3.2)	local	Modelo local sem custo API
Frontend	HTML + CSS + JS	—	Interface dark mode
Parsers	PyPDF2, docx, chardet	3.0.1	Processamento de arquivos

4. MODELOS UTILIZADOS

4.1 Modelo de Embeddings: all-MiniLM-L6-v2

- Tipo: Sentence Transformer baseado em BERT
- Dimensões: 384
- Tamanho: ~22M parâmetros
- Vantagem: Execução local, sem custo de API, balanceia velocidade e qualidade

4.2 Modelo Generativo: Ollama (llama3.2)

- Execução: 100% local, sem chamadas a API externas
- Temperatura: 0.2 (respostas determinísticas)
- Motivos: Gratuito, offline, privado, fácil instalação

5. VECTOR STORE: FAISS

5.1 Características

- Tipo: Biblioteca de busca vetorial da Meta AI Research
- Persistência: Índice binário em disco com metadados JSON
- Indexação: IndexFlatIP para produto interno normalizado (cosine similarity)

5.2 Vantagens

- Wheel pré-compilado para Windows/Linux/Mac
- Alta performance: bilhões de vetores com busca em milissegundos
- Biblioteca de referência (usada no Instagram, Pinterest)
- Código aberto pela Meta AI Research

6. FLUXO RAG IMPLEMENTADO

6.1 Pipeline Completo

O pipeline RAG executa as seguintes etapas de forma integrada:

12. Ingestão: Carregamento e chunking de documentos
13. Embedding: Geração de vetores para chunks
14. Armazenamento: Persistência em FAISS
15. Query: Busca de chunks relevantes
16. Geração: Criação de resposta com contexto

6.2 Estratégia de Chunking

- Tamanho do chunk: 600 caracteres
- Sobreposição: 80 caracteres
- Divisão: Prioritiza parágrafos
- Filtro: Chunks com menos de 20 caracteres são descartados

6.3 Configuração do Prompt

O prompt de sistema instrui o modelo a:

17. Responder APENAS com base no contexto fornecido
18. Indicar quando informação não está nos documentos
19. Citar as fontes ao final
20. Usar linguagem acessível mas precisa

7. LIMITAÇÕES DA SOLUÇÃO

Limitação	Descrição	Impacto
Dependência do Ollama local	llama3.2 precisa estar instalado	Alto
PDFs não-OCR	PDFs digitalizados não são lidos	Baixo
Sem histórico persistente	Contexto não salvo entre sessões	Médio
Limite de 20MB	Arquivos maiores são rejeitados	Baixo
Português vs Inglês	Embeddings treinados em inglês	Médio
Sem reranking	Busca pode trazer chunks irrelevantes	Médio
Alucinação do LLM	Modelo pode gerar informações incorretas	Alto

8. MELHORIAS FUTURAS

8.1 Técnicas Avançadas

- Reranking com Cross-Encoder para melhorar relevância
- Embeddings em português para melhor qualidade em PT-BR
- Chunking semântico dividindo por sentido
- Hybrid search combinando semântica com BM25

8.2 Infraestrutura

- Banco de dados PostgreSQL + pgvector para escala
- Autenticação JWT para múltiplos usuários
- Cache Redis para embeddings e respostas frequentes
- Streaming de respostas token por token

8.3 Experiência do Usuário

- Histórico persistente de conversas
- Feedback de relevância (thumbs up/down)
- Exportação em PDF das respostas
- Visualização de chunks usados na resposta

9. CONCLUSÃO



O SpaceRAG demonstra com sucesso a implementação de uma arquitetura RAG completa, capaz de responder perguntas complexas sobre a nova economia espacial com base em documentos fornecidos pelo usuário.

A solução combina embeddings locais eficientes (sentence-transformers), armazenamento vetorial persistente (FAISS) e um modelo generativo local (Ollama llama3.2) em uma interface web moderna e acessível.

O uso de RAG supera abordagens de fine-tuning para este caso de uso por ser mais flexível, mais econômico e mais confiável. A escolha do Ollama garante privacidade e funcionamento totalmente offline.